openMSX Compilation Guide

Contents

1. Introduction
    1.1 New Versions of this Document
    1.2 Purpose
    1.3 Revision History
2. Preparation
    2.1 Build Tools
    2.2 Libraries
3. Getting the Source Code
    3.1 Released Version
    3.2 Git Clone
4. Binary for Local System
    4.1 Compilation
    4.2 Installation
5. Stand-alone Binary
    5.1 Compilation
    5.2 Installation
7. Next Steps
8. Contact Info

1. Introduction

1.1 New Versions of this Document

The latest version of the openMSX manual can be found on the openMSX home page:

http://openmsx.org/manual/

You can also use this URL to get up-to-date versions of the hyperlinks if you printed out this manual.

1.2 Purpose

This guide is about openMSX, the open source MSX emulator that tries to achieve near-perfect emulation by using a novel emulation model. You can find more information about openMSX on the openMSX home page.

If you just want to use openMSX, there is likely a pre-packaged version you can use. This document describes how you can compile openMSX from source, which is useful if:

- There is not yet a pre-packaged version for your favourite platform.
- You want to closely follow the latest developments.
- You want to make changes to openMSX.

If you only want to follow the latest developments, chances are an automatic build has been performed for your platform. Links to such development builds can be found on the openMSX website, at the bottom of the Download box.

If you need help compiling openMSX, please contact us. If you needed any modifications to make openMSX compile, please send those modifications to us, so we can make openMSX ever more portable.

1.3 Revision History

For the revision history, please refer to the commit log.

2. Preparation

Before you can start compiling openMSX, please make sure your system has all the necessary build tools installed, as well as the libraries openMSX depends upon. The following sections tell you how you can prepare your system.

Every section starts with generic instructions that apply to all platforms and compilers. At the end of each section, platform specific notes cover differences or additional steps for certain platforms or compilers.

2.1 Build Tools

For compilation, you need Python, a C++ compiler, and some compiler-specific programs. If you have compiled packages from source before, you probably have some of these installed already.

Python
: A compact and dynamic programming language. The latest version should work fine. Note: on Windows you may have to add the path to Python (e.g. `C:\Python`) to your `PATH` manually.

C++ compiler
: There are three compilers that are supported to build openMSX: GCC, clang and Visual C++. The GCC compiler builds openMSX on all supported platforms, clang (Xcode) is the default on macOS, while Visual C++ is the primary compiler for Windows.

GCC

For compilation with GCC, you need GNU Make and g++:

make
: GNU implementation of the Make tool. Make interprets rules that define how a project should be built. Version 3.80 or higher is required.

g++
: The GNU C++ compiler. At least version 12 is required to build openMSX.

clang

macOS

Install Xcode from Apple, which you can find in the Mac App Store. The openMSX build is not done inside the Xcode IDE, but it uses the SDK and command line tools that are provided by Xcode. Xcode 16.4 or higher is required.

Visual C++

To build with Visual C++ you need to have the latest version of it installed. Currently, this is Visual C++ 2022.

How to obtain the Visual C++ compiler:

Visual Studio Community
: This is a free (as in beer) IDE and development environment. It builds 32 and 64-bit binaries out of the box. Make sure you download the latest version.

To generate an MSI installation package, you also need:

WiX
: This is a tool that generates MSI installer packages. We recommend using version 3.0 or later. If you see a message that the "Votive" component won't install, that's not a problem, just continue.

2.2 Libraries

openMSX depends on the following libraries:

SDL
: Simple DirectMedia Layer version 2, a cross-platform library that provides low-level access to video, audio and input devices.

SDL_ttf
: Library that provides support for TrueType Fonts in SDL.

libpng
: Library for handling PNG images.

zlib
: Library for file compression.

Tcl
: The Tool Command Language, an embeddable scripting language. openMSX requires Tcl 8.6; older versions will not work.

OpenGL
: Library for accelerated graphics. It is likely that OpenGL is already installed on your system. If not, you should probably get the OpenGL implementation from the manufacturer of your graphics card. On Linux, using Mesa with a DRI driver can be an alternative.

GLEW
: The OpenGL Extension Wrangler, a library that greatly simplifies the use of OpenGL extensions.

Ogg (optional)
: The Laserdisc emulation expects files in the Ogg container format for the video and audio of Laserdiscs.

Vorbis (optional)
: The Laserdisc emulation expects audio encoded with Vorbis in the Ogg container format for the audio of Laserdiscs. Vorbis was chosen since it is not patent-encumbered.

Theora (optional)
: The Laserdisc emulation expects video encoded with the Theora codec in the Ogg container format for the video of Laserdiscs. Again, Theora was chosen since it is not patent-encumbered.

ALSA (optional, Linux only)
: The ALSA library for ALSA MIDI out support on Linux.

GCC / clang

You can install the required libraries system-wide, or you can use the "3rd party libraries" support in the openMSX build system and build a stand-alone binary. System-wide is recommended if you run a modular operating system (such as Linux, BSD or macOS with MacPorts, Homebrew or Fink) and you intend to only use the openMSX binary on your computer. The 3rd party library system is recommended if you build for other operating systems (Windows, plain macOS or embedded systems) or if you want to build a binary that can be used on other computers besides your own. If you choose the latter, please skip to the Stand-alone Binary chapter.

Debian and Ubuntu Linux

You can easily install all required packages using the "build dependencies" feature of the APT package manager (if you have source packages in your package sources list):

```
sudo apt-get build-dep openmsx
```

Do note that if the Debian/Ubuntu package that is in the version of the distribution you are currently running is older than the openMSX version you are trying to compile (which is typically so), you might miss some dependencies that were added in the newer version. The build system will complain and you will have to install the remaining (missing) packages manually. Read on to get some tips about that.

Typical depending packages are: `libsdl2-ttf-dev libogg-dev tcl-dev libpng-dev libtheora-dev libvorbis-dev libsdl2-dev libglew-dev`. The complete list for Debian (which will be very very similar) is here.

Other Linux

Most Linux distributions have their libraries split over two packages: a runtime package and a development package. The runtime package for the "Foo" library is typically called `libfoo`, the development package is typically named `libfoo-dev` or `libfoo-devel`. Applications that use a library only require the runtime package to be installed, but compilation requires both the runtime package and the development package to be installed.

macOS

The easiest thing to do is to use the stand-alone binary method, which will get the libraries automatically.

If you want to do things manually, you can get the libraries from MacPorts, Homebrew or Fink. These tools provide access to a large collection of software packages or ports. You can use them to install those packages (or ports). The manual approach is not regularly tested by the openMSX developers; unless you enjoy tinkering with builds we suggest you use the stand-alone binary approach instead.

Visual C++

When building with Visual C++, the optional libraries mentioned above are treated as follows:

- Headers and libraries for OpenGL are included with Visual C++ by default.
- The GLEW, Ogg, Theora and Vorbis libraries are not optional, but required.

Also, when building with Visual C++, the "3rd party libraries" support in the openMSX build system must be used. The resulting binary statically links in all dependencies, including the C runtime. For more details, please see the Stand-alone Binary chapter.

3. Getting the Source Code

openMSX is developed using the tools GitHub freely offers to open source projects. The code is stored in Git, an open source version management system. Every once in a while, a new version of openMSX is released.

There are several options for getting the source code:

Released Version
: These are tested versions, which should little or no trouble getting them compiled and running. However, as openMSX development if often quite fast, these tested versions may not have all the latest features. Bugs that have been fixed since the last release could also still be present.

Git Clone
: Through Git you can get the same development version the openMSX developers are using. This is the bleeding edge version, which contains all the latest stuff and may be absolutely fantastic or horribly broken. Usually the latest openMSX code compiles and runs fine, but we're only human, so once in a while it may break. This version may also contain changes that have not been documented yet.

Releases are intended for general users, Git Clone is intended for (would-be) developers, heavy testers and people who want to follow up new developments closely. It might be a good idea to play with a tested release first. If you like what you see and want to get in deeper, you can switch to a Git Clone later. Using a Git Clone is quite comfortable, as it allows you to do efficient incremental updates, saving network bandwidth and compile time.

If the version you downloaded is either a lot older or a lot newer than this guide, it is a good idea to read the guide included in your downloaded version instead of the version you're reading right now. You can find the Compilation Guide in the directory `doc/manual`.

3.1 Released Version

You can download a released version of openMSX from the releases page on GitHub. The latest version is probably the best one. This guide assumes that you are using the latest release.

After downloading, type the following in a UNIX or MinGW Shell (in your start menu), or use another decompression tool:

```
tar xzvf openmsx-VERSION.tar.gz
```

in which `VERSION` is the openMSX version you downloaded, or use the file name you saved the tar.gz file with. The directory that is created by uncompressing the `tar.gz` file is called *the top of the source tree*.

3.2 Git Clone

Getting a Git clone means you use Git to retrieve the latest version of the source code of openMSX. This means you will need to install a Git client. This package is usually named `git`. There are graphical front-ends for Git, but this guide will tell you how to use Git from the command line. More information about Git can be found on the Git Documentation site.

Windows users might want to have a look at msysGit as a command line tool, TortoiseGit, which offers Windows Explorer integration, or Git Extensions, a tool which also includes Visual Studio integration.

The following command (which is also displayed when you browse an openMSX Git repository) retrieves the latest source code (also works on Windows when using msysGit):

```
git clone https://github.com/openMSX/openMSX.git openMSX
```

This line specifies where you want to retrieve the files from (host name of the Git server), what project you want to retrieve (*openMSX* in this case), what module you want to get (*openMSX.git* in this case, which is the module that contains the sources of the main openMSX program) and what directory it should be cloned to (we chose *openMSX* in this example).

If you're a developer, it makes sense to use this Git command line:

```
git clone git@github.com:openMSX/openMSX.git openMSX
```

Pro tip: read the GitHub docs about SSH keys if you want this to work smoothly, without having to (re)type your password all the time.

The Git command creates a directory called `openMSX` for you in the current directory (or in the directory you specified in TortoiseGit). In this manual, this directory is referred to as *the top of the source tree*. In addition to the openMSX code, you will see a hidden Git administration directory called `.git`. Do not mess with it (nor move contents of this directory around), otherwise Git will get very confused.

If you want to update your source tree later, go to the top of the source tree and type:

```
git pull
```

or right-click on the openMSX directory in Windows Explorer and select "TortoiseGit -> Pull...".

4. Binary for Local System

This section explains how to build openMSX for a local system. If you want to create a binary that runs everywhere, we recommend to skip this chapter and use the stand-alone binary method. This is also the recommended approach for macOS and especially for Microsoft Windows systems.

4.1 Compilation

Now that all the necessary tools and libraries are installed, you are almost ready to start the actual compilation of openMSX.

GCC / clang

The first thing you may want to know is that you can build openMSX in different flavours. The generic optimized flavour "opt" should work for all CPUs.

If you are testing new openMSX developments or making changes in openMSX, you can benefit from asserts and debug symbols. To get those, you should select the "devel" flavour, like this:

```
export OPENMSX_FLAVOUR=devel
```

Although the default flavours will probably be OK for most cases, you may want to write a specific flavour for your particular wishes. The flavour files are all named `build/flavour-*.mk`.

You can select the C++ compiler to be used by setting the `CXX` environment variable like this:

```
export CXX=g++-5
```

or:

```
export CXX=clang++
```

This can be useful if the default compiler on your system is an old version which is either not capable of compiling openMSX, or generates less efficient code.

Now we can let a script check if you have indeed all necessary libraries and headers installed. Go to the top of your openMSX source tree and run the following script:

```
./configure
```

This script will report what versions of libraries you have installed. It also reports which components can be built with those libraries. If the script reports that it can't build the openMSX core component, you should install the missing ones before you can continue. Otherwise, you can decide to install the libraries needed for the optional components, or to continue without building some components (e.g. the Laserdisc support).

If installing the correct libraries doesn't help, contact the openMSX developers. When filing a bug report, please attach the `probe.log` file created by the `configure` script in the directory `derived/<cpu>-<os>-<flavour>/config/`.

You can customise the build process by editing the file `build/custom.mk`. The thing you most likely might want to customise is the installation directory (`INSTALL_BASE`). If you are installing openMSX on a system on which you do not have superuser (root) privileges, you can set the installation directory to a subdirectory of your home directory.

After successfully running `configure`, it's finally time to compile everything. To start compilation, type:

```
make
```

Depending on how fast your system is, this may take anywhere between several minutes to half an hour.

If you get errors during compilation, there may be something wrong that was not detected by configure. Verify that you installed all required libraries, both the runtime and development packages. If that doesn't help, or we forgot to list a library openMSX depends on, please contact the openMSX developers. Make sure to include the error message you got.

If you want to debug openMSX compilation problems yourself, you can add `V=1` (verbose) to the Make command line to see all build commands as they are executed.

macOS

The version of Xcode required also determines which version of macOS you need to build openMSX out of the box.

Visual C++

While it is possible to configure the supplied Visual C++ project files to dynamically link against import libraries, this is not currently supported.

4.2 Installation

GCC / clang

Linux and BSD

To install openMSX, run the following command:

```
make install
```

This installs openMSX in `/opt/openMSX` by default. Note that only root has rights to write to system-wide directories such as `/opt`, so you may have to do `su` before `make install`, or use `sudo`:

```
sudo make install
```

macOS

On macOS, the build creates an application folder. You can "install" this simply by copying this folder to a different location. The application folder will depend on system-wide installed libraries, so it will not work on Macs lacking those libraries.

You can run openMSX from the application folder with the following command:

```
open derived/<cpu>-darwin-<flavour>/bindist/openMSX.app
```

If you want to see the messages openMSX prints to stdout and stderr, start openMSX like this:

```
derived/<cpu>-darwin-<flavour>/bindist/openMSX.app/Contents/MacOS/openmsx
```

5. Stand-alone Binary

This chapter describes how to build a binary of openMSX that does not depend on any libraries except those that are available on the platform by default. This procedure is highly recommended on Microsoft Windows. It is also recommended on macOS.

The stand-alone binary is made by linking statically against all libraries which are not available as part of the basic platform. The build system will automatically download the sources for these libraries and build them in the minimal configuration needed by openMSX.

5.1 Compilation

GCC / clang

If you want to change something about this process, for example switch to a newer library release, edit the `build/3rdparty.mk` Makefile.

Similar to standard compilation, you can set environment variables such as `OPENMSX_FLAVOUR`, `CXX`, `OPENMSX_TARGET_CPU` and `OPENMSX_TARGET_OS`.

To build a stand-alone binary, run the following command at the top of the source tree:

```
make staticbindist
```

The final output files will be produced in `derived/<cpu>-<os>-<flavour>-3rd/bindist`.

macOS

The final output is a DMG file (Mac disk image) containing the openMSX application folder and documentation. This file is internet-enabled, which means it will be automatically mounted after it is downloaded.

If you have an Apple Developers ID, you can automatically sign the app bundle by providing the following environment variable in your shell before building the app bundle:

```
export CODE_SIGN_IDENTITY=<your Apple Application Developer ID here>
make staticbindist
```

To find the exact string needed for the CODE_SIGN_IDENTITY, use the command:

```
security find-identity
```

Check if the code signing was successful with:

```
codesign -dv --verbose=4 derived/<cpu>-<os>-<flavour>-3rd/bindist/openMSX.app
```

Visual C++

When building with Visual C++, the result is a static executable with minimal dynamic library dependencies. Two platforms are supported:

- **Win32** - a 32-bit binary that runs on both 32 and 64-bit versions of Windows
- **x64** - a native 64-bit binary that runs only on 64-bit versions of Windows

Three different configurations are supported:

- **Debug**: assertions enabled, debug prints enabled, no optimization, very slow
- **Developer**: assertions enabled, debug prints disabled, no optimization
- **Release**: assertions disabled, debug prints disabled, full optimization

When building with Visual C++, you can either use the IDE or build from the command line using `msbuild`. To do the latter, you need to open a Visual Studio command prompt. A shortcut to a Visual Studio command prompt can usually be found in your start menu. For Visual Studio Community 2017, it will be in "Microsoft Visual Studio 2017\Visual Studio Tools" as the "Visual Studio Command Prompt".

In order to build openMSX, the libraries it depends on need to be downloaded and unpacked:

- Open a command prompt and go to the *top of the openMSX source tree*
- Type `python build\thirdparty_download.py windows`

The following steps can then be used to build openMSX:

- Open a Visual Studio command prompt
- Go to the *top of the openMSX source tree*
- To build 3rd party libraries, type `msbuild -p:Configuration=Release;Platform=x64 build\3rdparty\3rdparty.sln`
- To build openMSX, type `msbuild -p:Configuration=Release;Platform=x64 build\msvc\openmsx.sln`

The openMSX executable will be generated in `derived\x64-VC-Release\install`.

To build for other platforms or configurations, simply replace "Release" and "x64" in the command lines above with the desired options.

To build using the Visual C++ IDE, simply open the aforementioned solution files and from the Build menu select "Build Solution". This is the same as building from the command line using `msbuild`.

5.2 Installation

Visual C++

The following step creates .zip and .msi installation packages for openMSX on Windows using WiX:

- Build openMSX and wxCatapult for a given platform (Win32 or x64) and configuration (usually Release builds)
- Go to the *top of the openMSX source tree*
- Run `build\package-windows\package.cmd` **`platform configuration catapult_source_path`**:
  - **platform** is one of { Win32, x64 }
  - **configuration** is one of { Release, Developer, Debug }
  - **catapult_source_path** is the path to the wxCatapult directory

An example command line (for 64 bit) could be:

```
build\package-windows\package.cmd x64 Release ..\wxCatapult
```

The resulting package files can be found in `derived\x64-VC-Release\package-windows`.

7. Next Steps

If all went well, you should have openMSX installed now or have a stand-alone working binary. You can test it by executing openMSX from the command line:

```
openmsx
```

or, by double-clicking the openMSX executable or icon that resulted from the previous step.

You should get a screen similar to this:

```
C-BIOS 0.25        cbios.sf.net

No cartridge found.

This version of C-BIOS can
```

```
only start cartridges.
Please restart your MSX
(emulator) with a cartridge
inserted.
```

C-BIOS MSX2+ is the default system BIOS used by openMSX. It was written from scratch by BouKiCHi, who was kind enough to let us distribute it together with openMSX. It is not perfect yet, but it runs many ROM games well. Nowadays C-BIOS is a separate (SourceForge.net) project, with its own web page.

If you have a ROM image ready, you can try to run it with C-BIOS:

```
openmsx ~/msx/games/my-favourite-game.rom
```

or, you can just drop it on the openMSX executable.

The next step would be to read the openMSX Setup Guide. That document describes how you can configure openMSX to emulate actual MSX machines, such as the Panasonic FS-A1GT (turboR). It also describes how you can have openMSX start up with your personal settings, how you can configure openMSX and your system for optimal performance and several other configuration related topics. And finally there is of course the openMSX User's Manual, which describes all the things you can do with openMSX once it is fully running.

If you got stuck somewhere in the compilation and installation process, please contact us. The next chapter will tell you how.

macOS

You can run the openMSX application folder from Finder or from the command line:

```
open derived/<cpu>-darwin-<flavour>-3rd/bindist/openMSX.app
```

8. Contact Info

Since openMSX is still under heavy development, feedback and bug reports are very welcome!

Should you encounter problems, you have several options:

1. Go to our IRC channel: `#openMSX on libera.chat` and ask your question there. Also reachable via webchat! If you don't get a reply immediately, please stick around for a while, or use one of the other contact options. The majority of the developers live in time zone GMT+1. You may get no response if you contact them in the middle of their night...
2. Post a message on the openMSX forum on MRC.
3. Create a new issue in the openMSX issue tracker on GitHub. You need a (free) log-in on GitHub to get access.
4. Contact us and other users via one of the mailing lists. If you're a regular user and want to discuss openMSX and possible issues, join our `openmsx-user` mailing list. If you want to address the openMSX developers directly, post a message to the `openmsx-devel` mailing list. More info on the openMSX mailing lists, including an archive of old messages, can be found at SourceForge.

In all cases, please provide as much information as possible when you describe your bug or request.

GCC / clang

For experienced users: if you get a crash or a hang, try to provide a `gdb` backtrace. This will only work if you did not strip the openMSX binary of its debug symbols.

Another useful thing to do is to install the debug versions of libstdc++ and libc6, and then run openMSX with an `LD_LIBRARY_PATH=/usr/lib/debug` exported in the environment. This will give a more

detailed stack trace, especially in optimized code.

Visual C++

For experienced users: if you get a crash or a hang, try to provide a user dump. This will work for any openMSX binary, including pre-built binaries obtained from openmsx.org.

On Windows you can find user dump files for crashed processes in the "%LocalAppData%\CrashDumps" directory. The default Windows crash dump behaviour can be further customized as per MSDN.

To generate a user dump on demand on any Windows OS, please read KB286350.